



Qualitätsbegriff und Qualitätsmodelle

Programm für heute



QA|WARE

Vorlesung

1. Exkurs: Docker
2. Einschlägige Normen ISO 25000 / ISO 9000
3. Qualität im Spannungsfeld
4. Das Kano Modell als Entscheidungshilfe
5. GQM als Tool zur Entwicklung eigener Qualitätsmodelle

Übung

1. QAware OpenSource Quickcheck durchführen
2. Vorstellung der Ergebnisse in Kleingruppen (ca. 5min)
3. Übung zur GQM
4. Vorstellung der Ergebnisse in Kleingruppen (ca. 5min)



QA|WARE

Exkurs: Betriebssystem-Virtualisierung mit Docker

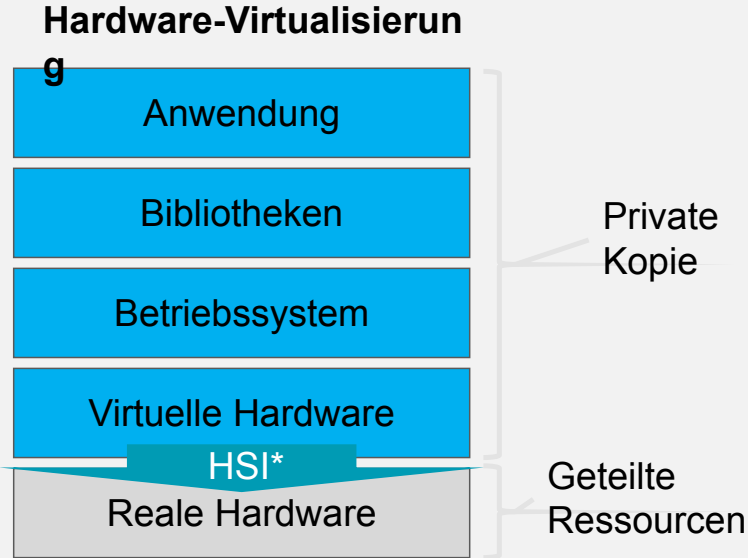
Docker

Docker ist eine Automationsumgebung für Betriebssystem-Virtualisierung.

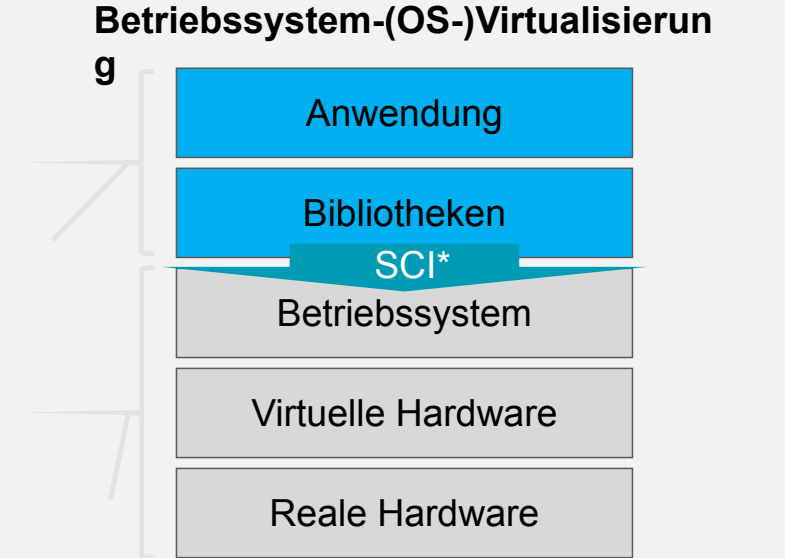
Aktuell unterstützt Docker Linux als Host-Betriebssystem. Seit Windows Server 2016 steht nun auch eine Windows-Variante zur Verfügung.

Docker ist als Werkzeug eines Cloud-Anbieters entstanden und ist mittlerweile eines der sichtbarsten und aktivsten Open-Source-Ökosysteme.

Hardware- vs. Betriebssystem-Virtualisierung



- Stärkere Isolation
- Höhere Sicherheit



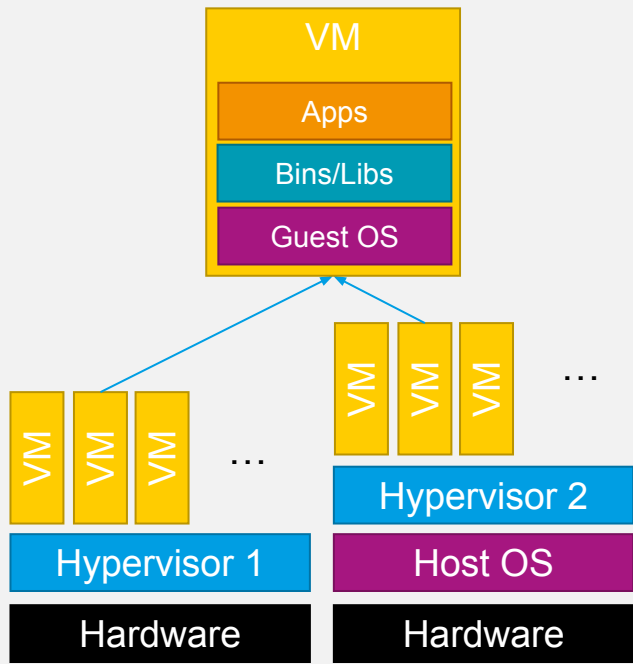
- Geringeres Volumen der privaten Kopie
- Geringerer Overhead
- Kürzere Startup-Zeit

*) HSI = Hardware Software Interface
SCI = System Call Interface

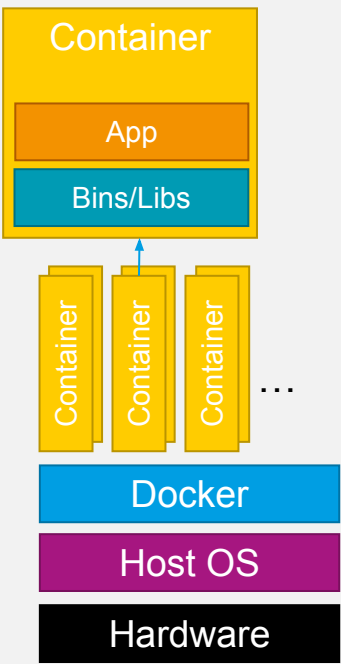
Containerization mit Docker



QAWARE



Type 1 / Type 2 Virtualization

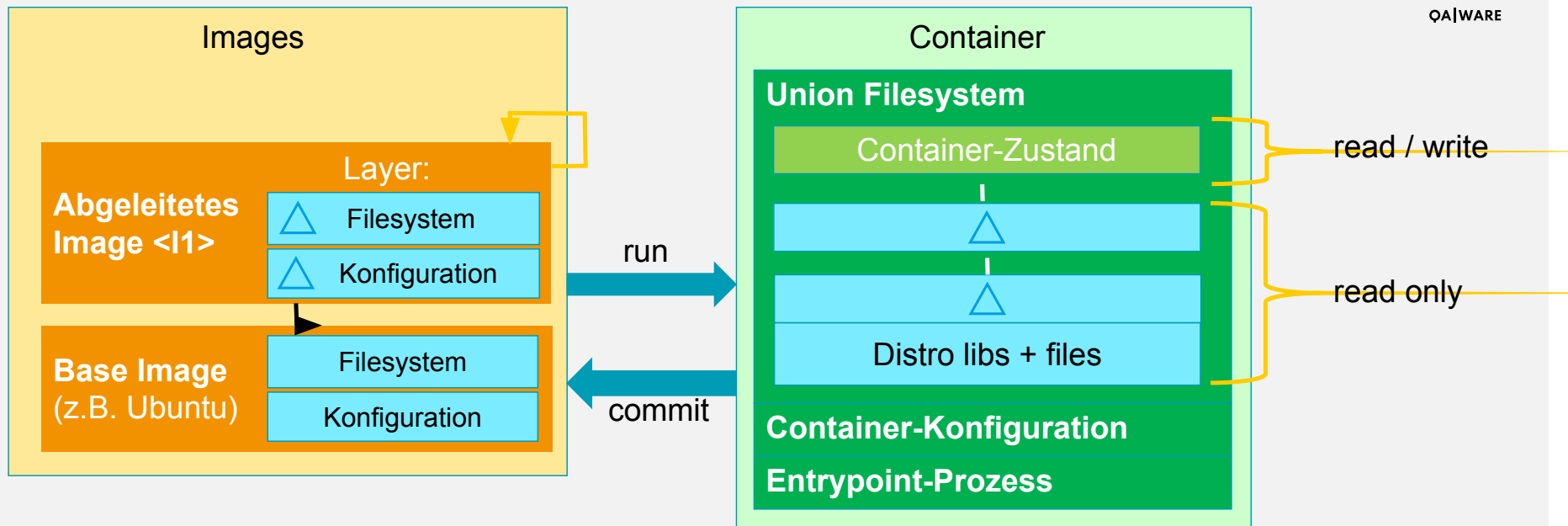


Containerization

Im Zentrum von Docker stehen Images und Container.



QAWARE



Ruhender und transportierbarer Zustand

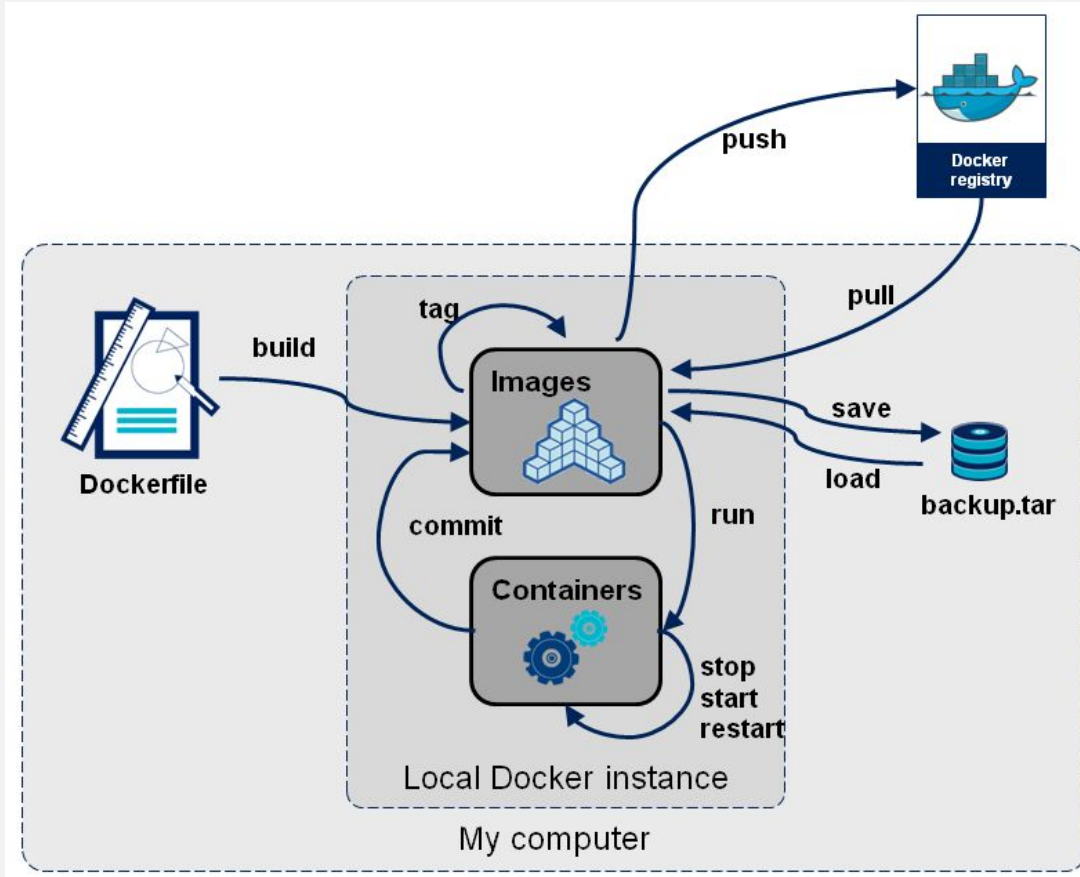
Laufender Zustand

Ein Container läuft so lange wie sein Entrypoint-Prozess im Vordergrund läuft. Docker merkt sich den Container-Zustand.

Der Docker Workflow.



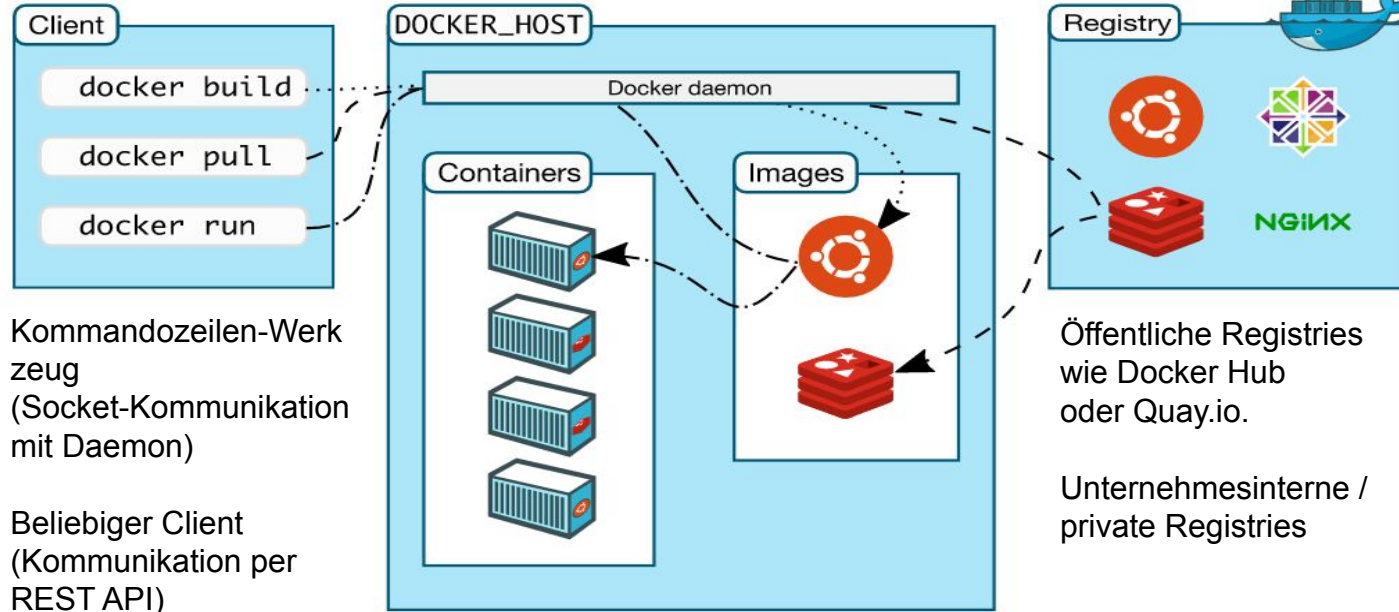
QA|WARE



Die Docker-Architektur.



QAWARE

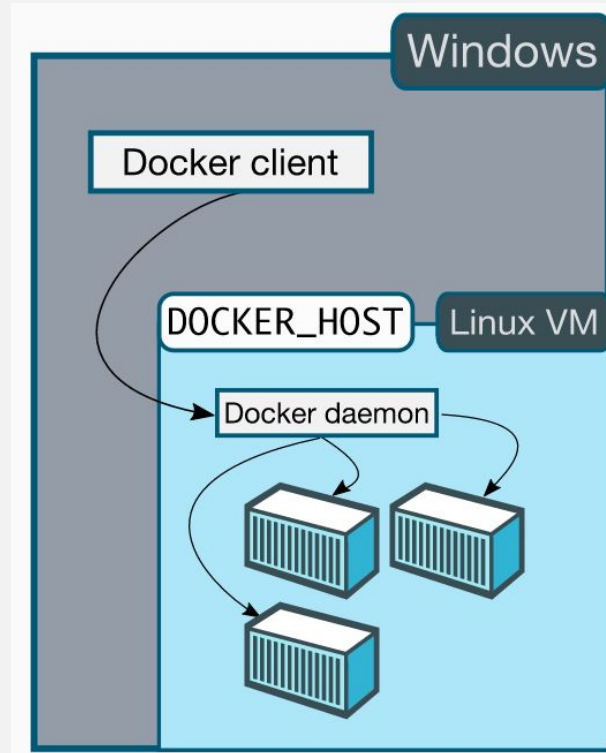


Der Docker-Daemon ist die zentrale Steuerungseinheit und läuft direkt als Prozess im Host-Betriebssystem. Er verwaltet alle lokalen Container und Images auf dem Host.

Zusammenspiel zwischen Host und Docker Machine.



QA|WARE



hub.docker.com ist die öffentliche Standard-Registry für Docker-Images.



QA|WARE

Explore Help

solr Sign up Log In

Repositories (555)

All

 solr official	280 STARS	100K+ PULLS	DETAILS
 anapsix/solr public automated build	1 STARS	8.0K PULLS	DETAILS
 apopelo/solr public automated build	1 STARS	1.6K PULLS	DETAILS
 ckan/solr public automated build	1 STARS	896 PULLS	DETAILS
 harisekhon/solr public automated build	0 STARS	6.6K PULLS	DETAILS
 makuk66/docker-solr public automated build	81 STARS	50K+ PULLS	DETAILS

Provisionierung von Images mit dem Dockerfile



QA|WARE

Das Dockerfile definiert Aufbau und Inhalt des Image.

My Image

Layer 8

Layer 7

Layer 6

Layer 5

Layer 4

Layer 3

Layer 2

Layer 1

Niemals „latest“ verwenden. Antipattern

FROM qaware/alpine-k8s-ibmjava8:8.0-3.10

LABEL maintainer="QAware GmbH <qaware-oss@qaware.de>"

RUN mkdir -p /app

COPY build/libs/zwitscher-service-1.0.1.jar /app/zwitscher-service.jar

COPY src/main/docker/zwitscher-service.conf /app/

ENV JAVA_OPTS -Xmx256m

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "/app/zwitscher-service.jar"]

Dockerfile-Kommandos

Element	Meaning
FROM <image-name>	Sets to base image (where the new image is derived from)
MAINTAINER <author>	Document author
RUN <command>	Execute a shell command and commit the result as a new image layer (!)
ADD <src> <dest>	Copy a file into the containers. <src> can also be an URL. If <src> refers to a TAR-file, then this file automatically gets un-tared.
VOLUME <container-dir> <host-dir>	Mounts a host directory into the container.
ENV <key> <value>	Sets an environment variable. This environment variable can be overwritten at container start with the <code>-e</code> command line parameter of docker run .
ENTRYPOINT <command>	The process to be started at container startup
CMD <command>	Parameters to the entrypoint process if no parameters are passed with docker run
WORKDIR <dir>	Sets the working dir for all following commands
EXPOSE <port>	Informs Docker that a container listens on a specific port and this port should be exposed to other containers
USER <name>	Sets the user for all container commands

Typische Kommandos eines Docker Workflows

Command	Action
<code>docker build -t <image> .</code>	Build Docker image with given tag in current directory
<code>docker images</code>	Prints all local images
<code>docker run</code> <code>-d</code> <code>-v <volume mounts></code> <code>-p <host-port>:<container-port></code> <code><image> <entrypoint process></code>	Run a Docker image: Creates and runs a container. ▪ in background ▪ with host directory mounted into the container ▪ with port forwarding from host to container ▪ image name (and optional entrypoint process)
<code>docker run</code> <code>-ti</code> <code><image> /bin/sh</code>	Run a Docker image and open a shell within the container ▪ ... with forwarding of local terminal ▪ Image name and shell (or „/bin/bash“)
<code>docker ps -a</code>	Prints all containers (without <code>-a</code> = only running containers)
<code>docker commit <container> qaware/foo</code>	Store container as local image
<code>docker kill <container></code> <code>docker rm <container></code>	Terminate container (send SIGKILL to entrypoint process) Remove container
<code>docker rmi -f <image></code>	Remove local image

Hilfreiche Kommandos für Container Troubleshooting



QAWARE

Command	Action
<code>docker inspect <container></code>	Shows container metadata (e.g. IP)
<code>docker logs <container></code>	Prints container syslog
<code>docker top <container></code>	Prints all running processes within a container (like <code>ps -a</code> within the container)
<code>docker exec -ti <container> /bin/sh</code>	Connect terminal to running container
<code>docker stats <container></code>	Shows container runtime statistics (e.g. CPU usage, IO intensity, ...)

Einfache Containerorchestrierung mit Docker Compose.



QA|WARE

```
version: '3.3'

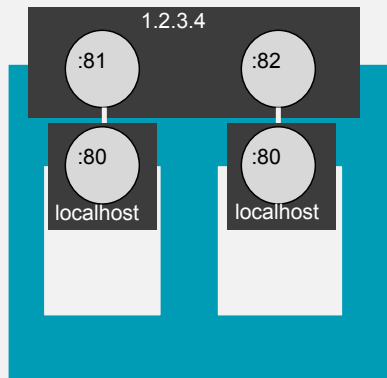
services:
  db:
    image: mysql:8.0.29-oracle
    container_name: sqs-db
    environment:
      - MYSQL_ALLOW_EMPTY_PASSWORD=yes
      - MYSQL_ROOT_PASSWORD=dasdlDDX1!asd;odYY
      - MYSQL_ROOT_HOST=%
      - MYSQL_DATABASE=sqs-db
    ports:
      - "3308:3306"

  mail:
    image: ...
```

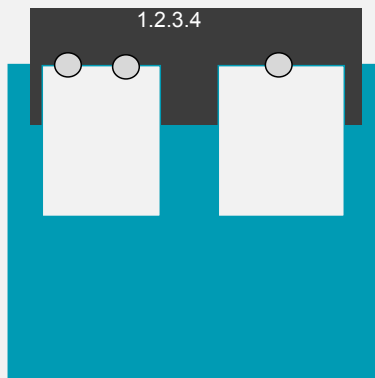
Die Docker Networking Modes.



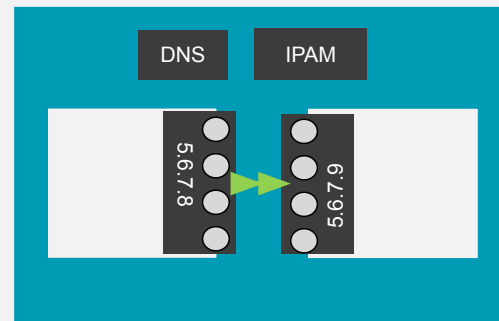
QAWARE



Bridge



Host



Overlay Network

`docker network ls`

`docker network inspect bridge`

`docker network create --driver overlay`

`multi-host-network`

`docker network connect multi-host-network container1`

- Bound
- port
- Guest
- Host

https://github.com/veggiemonk/awesome-docker



QA|WARE

This repository

Search

Pull requests

Issues

Gist

veggiemonk/awesome-docker

Watch · 406

Star · 4,796

Fork · 536

Code

Issues 0

Pull requests 0

Projects 0

Wiki 0

Pulse

Graphs

A curated list of Docker resources and projects

http://veggiemonk.github.io/awesome-docker/

801 commits

4 branches

1 release

138 contributors

Create new file · Upload files · Find file

Clone or download

Branch master · New pull request

gveggiamonk committed on GitHub Merge pull Latest commit Bff5333 a day ago

UPDATE README from xrayl0p@patch-2

• attione attione 10 months ago

• travis.vml Update travis.vml 28 days ago

• CONTRIBUTING.md fix contribution info 6 months ago

• LICENSE.md Create License.md a year ago

• README.md Added 'Where to start - on Windows' section 3 days ago

• README.md

Awesome Docker

A curated list of Docker resources and projects inspired by @brendanruff/awesome and improved by these amazing contributors.

It's now a GitHub project because it's considerably easier for other people to edit, fix and expand on Docker using GitHub. Just click README.md to submit a pull request. If this list is not complete, you can contribute to make it so.

Please, help organize these resources so that they are easy to find and understood for new comers. See how to Contribute for tips!

If you see a link here that is not (any longer) a good fit, you can fix it by submitting a pull request to improve this file. Thank you!

The creators and maintainers of this list do not receive and should not receive any form of payment to accept a change made by any contributor. The goal of this repo is to index articles, learning materials and projects, not to advertise for profit. All pull requests are merged by default and removed if inappropriate or unavailable, or fixed when necessary.

All the links are monitored and tested with awesome_bot made by @dhamming

What is Docker ?

Docker is an open platform for developers and sysadmins to build, ship, and run distributed applications. Consisting of Docker Engine, a portable, lightweight runtime and packaging tool, and Docker Hub, a cloud service for sharing applications and automating workflows, Docker enables apps to be quickly assembled from components and eliminates the friction between development, QA, and production environments. As a result, IT can ship faster and run the same app, unchanged, on laptops, data center VMs, and any cloud.

Source: What is Docker

Where to start ?

10-minute Interactive Tutorial

Docker Training

Read this complete article: Basics - Docker, Containers, Hypervisors, CentOS

Watch the video: Docker for Developers (54:26) by @jpetazzo

Docker Jumpstart: a quick introduction

Docker Curriculum: A comprehensive tutorial for getting started with Docker. Teaches how to use Docker and deploy dockerized apps on AWS with Elastic Beanstalk and Elastic Container Service.

Install Docker on your machine and play with a few Useful Images

Try Panamax: Docker Management for Humans & will install a CoreOS VM with VirtualBox and has nice front end

Install Docker Toolbox Docker: Toolbox is an installer to quickly and easily

- Docker Containers on the desktop by @frazzle! The funniest way to learn about docker! (Tips: checkout her profiles and her dockerfiles)
- Container basics and fun images by @frazzle! @ DockerCon 2015 MUST WATCH VIDEO (38:50)
- Learn Docker Full environment set up, screenshots, step-by-step tutorial and more resources (video, articles, cheat sheets) by @dnyl
- Docker Caves: What You Should Know About Running Docker in Production (written 11 APRIL 2016) MUST SEE
- How to Whole Learn Docker in your web browser, no setup or installation required.
- Docker for all - Developers, Testers, DevOps, Product Owners - Videos Docker Training Videos for all

Where to start - on Windows ?

- Windows Containers Quick Start Overview of Windows containers, drilling down to Quick Starts for Windows 10 and Windows Server 2016
- Build And Run Your First Docker Windows Server Container Walkthrough installing Docker on Windows 10, building a Docker image and running a Windows container
- Video: Windows Containers and Docker: The 101 A 20-minute overview, using Docker to run PowerShell, ASP.NET Core and ASP.NET apps
- A Comparative Study of Docker Engine on Windows Server vs Linux Comparing the feature sets and implementations of Docker on Windows and Linux
- Docker with Microsoft SQL 2016 + ASP.NET Demonstration running ASP.NET and SQL Server workflows in Docker
- Running a Legacy ASP.NET App in a Windows Container Steps for Dockerizing a legacy ASP.NET app and running as a Windows container
- Exploring ASP.NET Core with Docker in both Linux and Windows Containers Running ASP.NET Core apps in Linux and Windows containers, using Docker for Windows

MENU

- What is Docker ?
- Where to start ?
- MENU
- Useful Articles
 - Main Resources
 - General Articles
 - Deep Dive
 - Networking
 - Metal
 - Multi-Server
 - Cloud Infrastructure
 - Good Tips
 - Newsletter
 - Continuous Integration
 - Optimizing Images
 - Service Discovery
 - Security
 - Performances
 - Raspberry Pi & ARM
 - Other
- Books
 - In English
 - Chinese
 - German
 - Portuguese
- Tools
 - Terminal User Interface
 - Dev Tools
 - Continuous Integration / Continuous Delivery
 - Deployment
 - Hosting for repositories (registries)

- Reverse Proxy
- Web Interface
- Local Container Manager
- Volume management and plugins
- Useful Images
- Dockerfile
- Docker Compose file
- Storing Images and Registries
- Monitoring
- Networking
- Logging
- Deployment and Infrastructure
- PaDs
- Remote Container Manager / Orchestration
- Security
- Service Discovery
- Metadata
- Sides
- Videos
 - Main Account
 - Useful videos
- Interactive Learning Environments
- Interesting Twitter Accounts
- People
- Communities and Meetups

Useful Articles

Main Resources

- Docker Weekly Huge resource
- Docker Cheat Sheet by @swargent MUST SEE
- Docker Printable Refcard by @dmonmond
- CenturyLink Labs
- Valuable Docker Links Very complete
- Docker Ecosystem (Mind Map) MUST SEE
- Docker Ecosystem (PDF) MUST SEE find it on blog by Brynglarer Peter.
- Blog of @frazzlezell
- Blog of @jpetazzo
- Blog of @progrium
- Blog of @jwilder
- Blog of @robertymchael
- Blog of @jdelabla
- Blog of @seebga
- Blog of @dcoffito
- Digital Ocean Community
- Container42
- Container solutions
- DockerOne Docker Community (in Chinese) by @LYingJie
- Project Web Dev : Article series: How to create your own website based on Docker
- Docker vs. VMs? Combining Both for Cloud Portability Nirvana
- Docker Containers on the desktop by @frazzle! The funniest way to learn about docker! (Tips: checkout her profiles and her dockerfiles!)
- Awesome Linux Container more general about container than this repo, by @Friz-zy.

General Articles

- Getting Started with Docker by @fideloper -- Servers For Hackers is valuable resource. At some point, every programmer finds themselves needing to know their way around a server.
- What Is Docker and how do you monitor it?
- How to Use Docker on OS X: The Missing Guide
- Docker for Java Developers

- Deploying NGINX with Docker
- Eight Docker Development Patterns
- Rails Development Environment for OS X using Docker
- Logging on Docker: What You Need to Know - see the video (~50min)
- Comparing Five Monitoring Options for Docker
- Minimalistic data-only container for Docker Compose (Written Mar 1, 2015)
- Running Docker Containers with Systemd
- Dockerizing Flask With Compose and Machine - From Localhost to the Cloud -- GitHub Learn how to deploy an application using Docker Compose and Docker Machine (written 17 April 2015)
- Why and How to use Docker for Development (written 28 APR 2015)
- Automating Docker Logging: ElasticSearch, Logstash, Kibana, and Logspout (written 27 APR 2015)
- Docker Host Volume Synchronization (written 1 JUN 2015)
- From Local Development to Remote Deployment with Docker Machine and Compose (written 2 JUL 2015)
- Docker: Build, Ship and Run Any App, Anywhere by Martin Downs, Wiebe van Gestel, Rik Nijssen, and Rick Wieman from Delft University of Technology (written 2 JUL 2015)
- Joining the Docker Ship Learn how to contribute to docker (written 9 JUL 2015)
- Continuous Deployment with Gradle and Docker Describes a complete pipeline from source to production deploy (includes a complete Spring Boot example project) by @svenlo
- Containerization and the Paad Cloud -- This article discusses the requirements that arise from having to facilitate applications through distributed multcloud platforms.
- Docker for Development: Common Problems and Solutions by @druhuas
- Docker Adoption Data A study by Datadog on the real world Docker usage statistics and deployment patterns.
- How to monitor Docker (4-part series)
- Using Ansible with Docker Machine to Bootstrap Host Nodes by @mathiasdole
- Swarn S. Fleet v. Kubernetes v. Mesos Comparing different orchestration tools. (written OCT 2015)
- The Shortest of Docker Hosting: There are so many specialized and optimized Docker hosting services available, it's high time for a review to see what's on offer (by Chris Ward).

Portuguese Articles

- Uma rápida introdução ao Docker e instalação no Ubuntu
- O que é uma imagem e o que é um container Docker?
- Criando uma imagem Docker personalizada
- Comandos mais utilizados no Docker

Deep Dive

- Creating containers - Part 1 This is part one of a series of blog posts detailing how docker creates containers. By @robertymchael
- Data-only container madness

Networking

- How to Route Traffic with Weave 0.10 (written 22 APR 2015)
- Using Docker Machine through a Tor Docker container by @frazzle (written 20 JUN 2015)
- Demystifying Docker overlay networking. By @nigelboulton

Metal

- How to use Docker on Full Metal
- CaringOS: A bare essential OS for running the Docker Engine on bare metal or Cloud.

Multi-Server

- A Docker based mini-PaaS by @progrium
- A multi-host scalable web services demo using Docker swarm, Docker compose, NGINX, and Blockbridge

Cloud infrastructure

- Cloud Infrastructure Automation for Docker Nodes

Good Tips

- 24 random docker tips by @csabapalfi
- GUI Apps with Docker by @frghm
- Automated Nginx Reverse Proxy for Docker by @jwilder
- Using XEster with Boot2Docker
- A Simple Way to Dockerize Applications by @jwilder
- Building good docker images by @bergrhoff
- 10 Things Not To Forget Before Deploying Docker in Production
- Docker CIFS - How to Mount CIFS as a Docker Volume
- Nginx Proxy for Docker (written 9 JUL 2015)
- Dealing with instant containers dependency in docker-compose by @brochobruno
- Docker tips by @jwilder
- Docker on Windows behind a Firewall by @kaltoutet
- Pulling Git into a Docker image without leaving SSH keys behind by @khaah
- 6 Million Ways To Log In Docker by @raychaser
- Dockerfile Generator (ruby script)
- Running Production Hadoop Clusters in Docker Containers
- 10 practical docker tips (Dec 2015) by @jdoediner
- Kubernetes Cheatsheet - A great resource for managing your Kubernetes installation
- Container Best Practices - Red Hat's Project Atomic created a Container Best Practices guide which applies to everything and is updated regularly.
- Production Meteor and Node Using Docker, Part 1 by @projectreactor
- Resource Management in Docker by @marcelgdmann

Newsletter

- Docker Team
- CenturyLink Labs
- Tutum
- Shipable
- WebOps weekly

Continuous Integration

- Docker and Phoenix: How to Make Your Continuous Integration More Awesome
- Jenkins 2.0 - Sonarcast Series by Virendra Bhalothia
- Pushing to ECR Using Jenkins Pipeline Plugin by @mkazis87

Optimizing Images

- Create the smallest possible Docker container
- Creating a Docker image from your code
- Optimizing Docker Images
- How to Optimize Your Dockerfile by @tutumcloud
- Building Docker Images for Static Go Binaries by @kyleisightower
- Squashing Docker Images by @jwilder
- Dockerfile Golf (or optimizing the Docker build process)
- DockerSlim shrinks fat Docker images creating the smallest possible images.
- Skinnywhale Skinnywhale helps you make smaller (as in megabytes) Docker containers.
- MicroBadger - Analyze the contents of images and add metadata labels to Docker containers.

Service Discovery

- @progrium Service Discovery articles series:
 - Consul Service Discovery with Docker
 - Understanding Modern Service Discovery with Docker
 - Automatic Docker Service Announcement with Registrar

QAware | 19



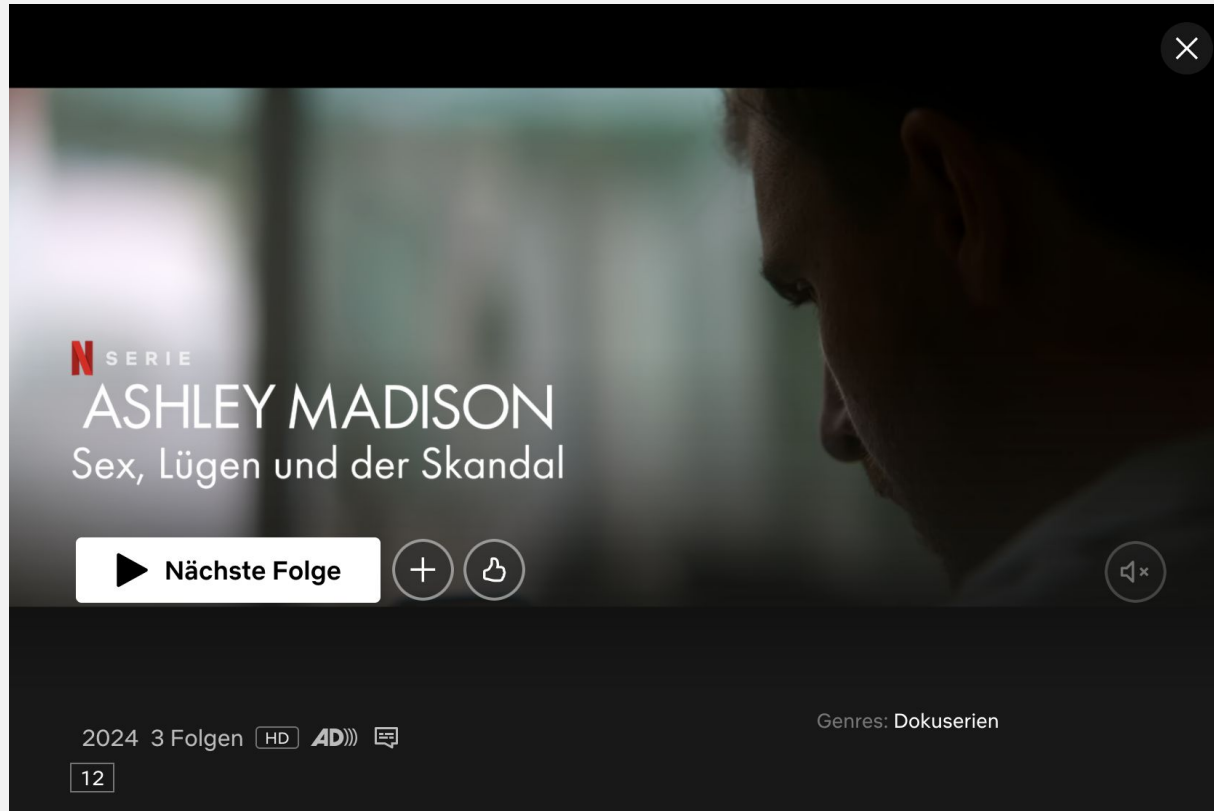
QA|WARE

Recap

Netflix Dokumentation zur letzten Veranstaltung.



QA|WARE



Nachträglich ist immer teurer als rechtzeitig.



QA|WARE



Der Ashley Madison Hack war für alle Beteiligten sehr “teuer”.



QA|WARE

- Pro Tag fehlen ca 100.000 USD Umsatz durch zahlende Kunden:innen
- Computerforensiker mussten bezahlt werden
- Indirekte Kosten durch:
 - Verlust von Jobs (CEO + Menschen, deren Daten leaked wurden)
 - Selbstmorde
 - Folgekosten (Software neu aufsetzen + Sicherheitslücken identifizieren)



QA|WARE

Qualitätskosten

Vorsorge ist besser als Nachsorge.



QA|WARE

- Beispiel Zahnarzt: Ein einfache Füllung kostet ca. 80 - 200 Euro (inklusive Zahnarzt und Material)
- Wird das Loch im Zahn allerdings nicht rechtzeitig behandelt, so kann die folgende Wurzelbehandlung zwischen 500 und 1500 euro betragen
- Dies entspricht einem Mehraufwand mit dem Faktor von ~17

Je früher wir Fehler erkennen, desto billiger ist es diese zu beheben.



QA|WARE

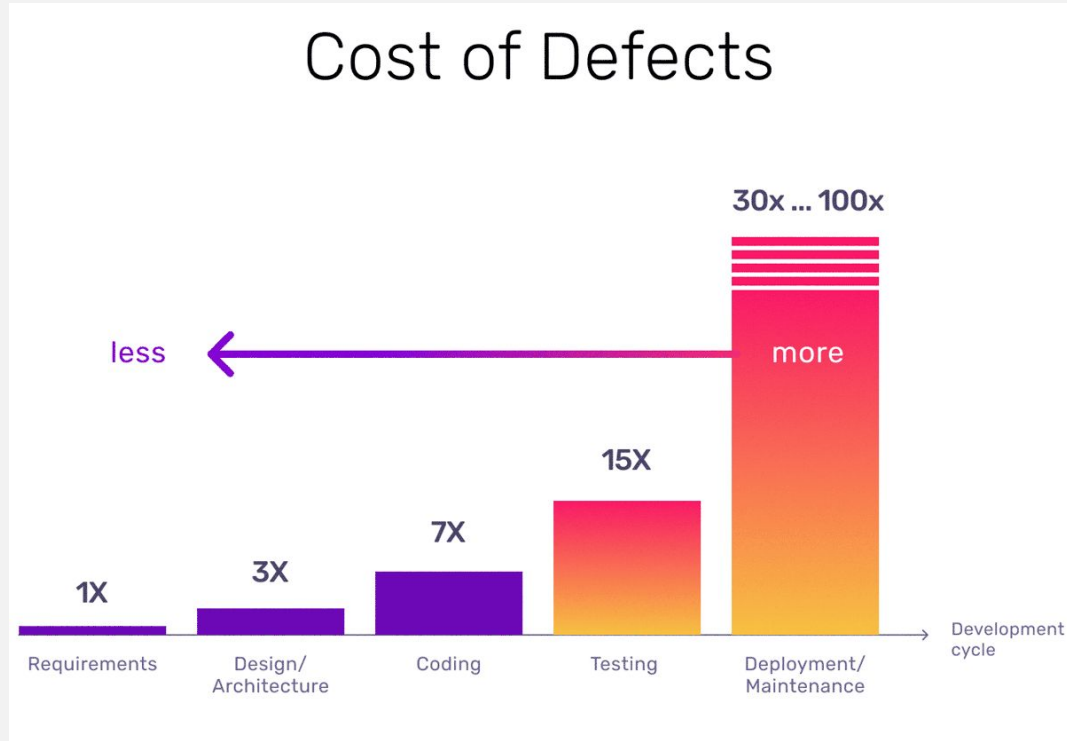
Welchen Unterschied macht es in Bezug auf die Kosten einen Fehler bereits bei der Anforderungsanalyse im Vergleich zur Live-Anwendung zu finden?



Je früher wir Fehler erkennen, desto billiger ist es diese zu beheben.



QA|WARE



Qualitätskosten sind real.



QA|WARE

- Das PSA-Portal der Telekom muss modernisiert werden, weil es ursprünglich nicht für die Menge an Anforderungsdokumenten designed war
- Dokumente können durch eine komplizierte und verwobene Struktur erstellt werden und werden überall im Code verwendet und dynamisch berechnet
- Macht das Portal an vielen Stellen (unnötig) langsam

Qualitätskosten durch technische Schulden im PSA Portal der Telekom.



Letztes Jahr standen wir mit dem PSA Portal der Telekom an genau dieser Stelle. Wir bewegen uns hier bereits in einer Live-Applikation d.h. Konzeptionsphase etc. sind lange vorbei. Unser Faktor ist also schon bei mindestens 30.

Wir haben einen Cache und eine Vorberechnung der Dokumente eingebaut.



QA|WARE

- Dokumente können aus zwei Sichten gesehen werden:
 - Autorensicht:
 - Herleitung und Struktur der Dokumente wichtig
 - Dokumente sind dynamisch und werden verändert
 - Dokumente müssen dynamisch berechnet werden
 - Lesersicht:
 - Nur finale Struktur der Dokumente wichtig
 - Dokumente ändern sich nicht mehr und sind statisch
 - Dokumente sind vorberechnet und können aus einem Cache bezogen werden

Dieser Umbau war zu diesem späten Zeitpunkt sehr teuer.



- Umbau zog sich über 2 Monate reiner Entwicklungszeit
- Bedingt durch schlechte Testabdeckung und hohen Komplexitätsgrad des Umbaus waren wir mindestens noch einmal 2 weitere Monate mit Bugfixing beschäftigt
- Kosten wurden zwischen Telekom und QAware aufgeteilt
- Gesamtkosten über 60.000 €

Die Gründe für die steigenden Kosten sind vielfältig.



QA|WARE

- Test- und Deploymentsetup ist bereits geschrieben und muss bei Änderungen im Zweifel komplett angepasst werden
- Je tiefliegender die Fehler sind, desto mehr andere Stellen sind von Änderungen betroffen
- Große Refactorings und Bugfixes produzieren eventuell neue Probleme
- Kleine Fehler an grundlegenden Komponenten oder anfänglich falsch getroffene Designentscheidungen führen oft zu massiven Problemen
- Entwickler:innen sind nicht mehr so tief im Thema und müssen sich “neu reindenken”



QA|WARE

Qualität in den Normen (kurze Zusammenfassung)

Die ISO 9000 beschreibt Qualitätsmanagementsysteme.



QA|WARE

- Fokus auf kontinuierliche Verbesserung und Kundenzufriedenheit
- Betrachtet die Interaktion und Wechselwirkung von Prozessen innerhalb eines Unternehmens
- Betont die Bedeutung von Dokumentation von Prozessen, um Konsistenz und Qualität zu gewährleisten
- Betont die Verantwortung des Managements für Implementierung und Aufrechterhaltung eines Qualitätsmanagementsystems.

Die ISO 25000 - SQuaRE: Software Product Quality Requirements and Evaluation.

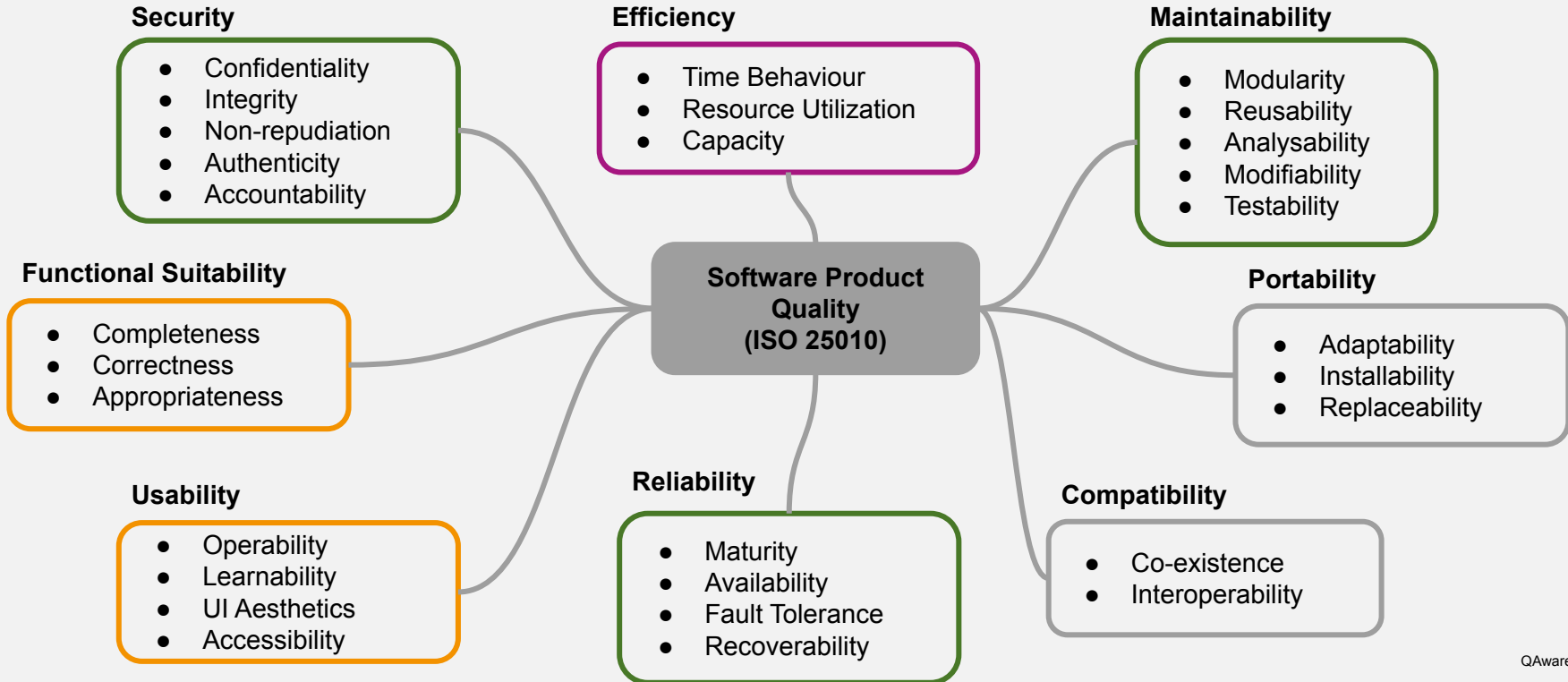


- Definiert Qualitätsmerkmale und Eigenschaften wie:
 - Funktionalität & Benutzbarkeit
 - Zuverlässigkeit & Wartbarkeit
 - Effizient
 - Portabilität
- Enthält Standards für Messung und Bewertung von Softwarequalität
- Bietet Richtlinien zur kontinuierlichen Verbesserung von Softwareprozessen
- Fokussiert sich auf die Erfüllung der Anforderungen und Erwartungen der Nutzer:innen

Der Qualitäts-Oktopus mit seinen “-ilities” hat Einfluss auf die 4 Sichten der Architektur eines Systems.



QA|WARE



Die ISO 25000 und die ISO 9000 setzen auf verschiedenen Ebenen an.



QA|WARE

Norm	ISO 9000	ISO 25000
Fokus	Qualitätsmanagement-systeme	Softwarequalität
Anwendungsbereich	Allgemeine Unternehmensprozesse	Softwareprodukte- und prozesse



QA|WARE

Qualität als Spannungsfeld

Softwarequalität im Projektalltag ist immer ein Kompromiss.

Aus Projektmanager:innensicht ein Kompromiss aus
welchen Stellgrößen?



Das Spannungsfeld lässt sich auf unterschiedliche Weise beschreiben.



QA|WARE

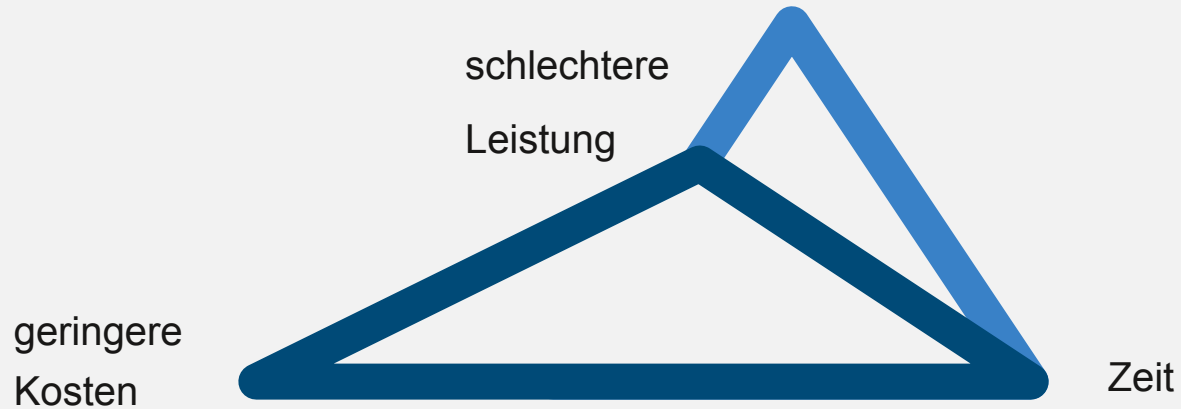
Das magische Dreieck



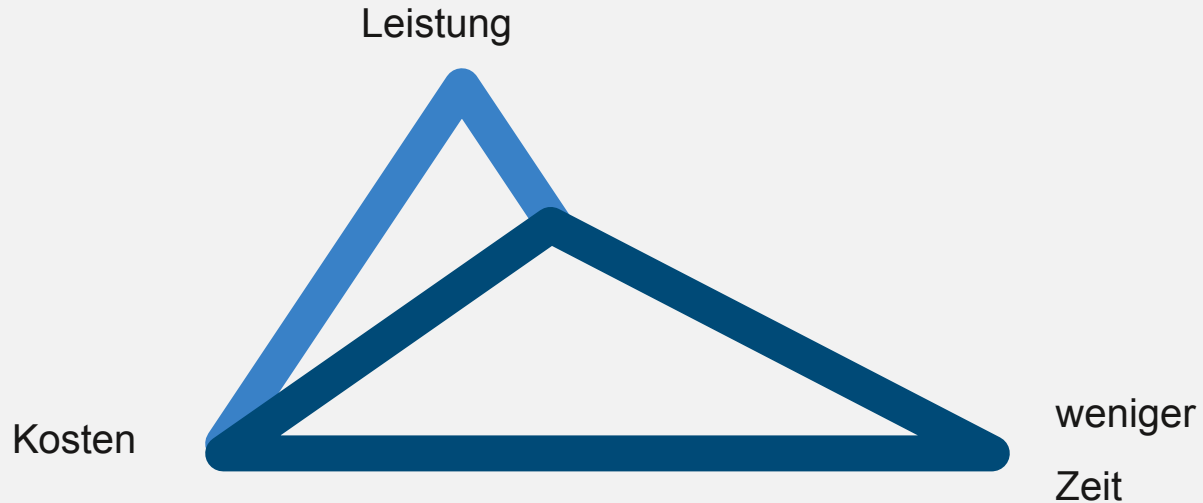
Das Spannungsfeld lässt sich auf unterschiedliche Weise beschreiben.



QA|WARE

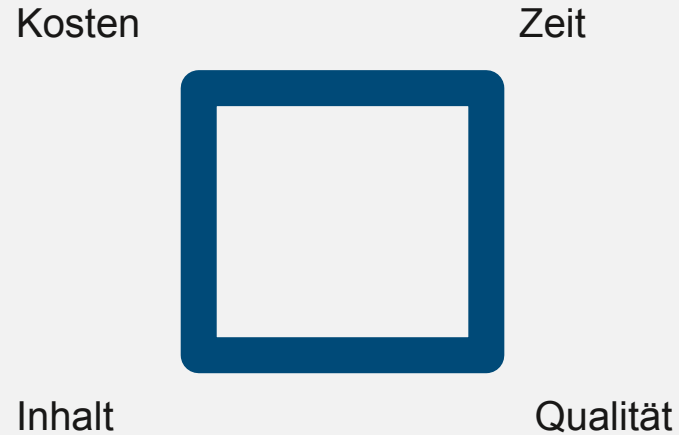


Das Spannungsfeld lässt sich auf unterschiedliche Weise beschreiben.

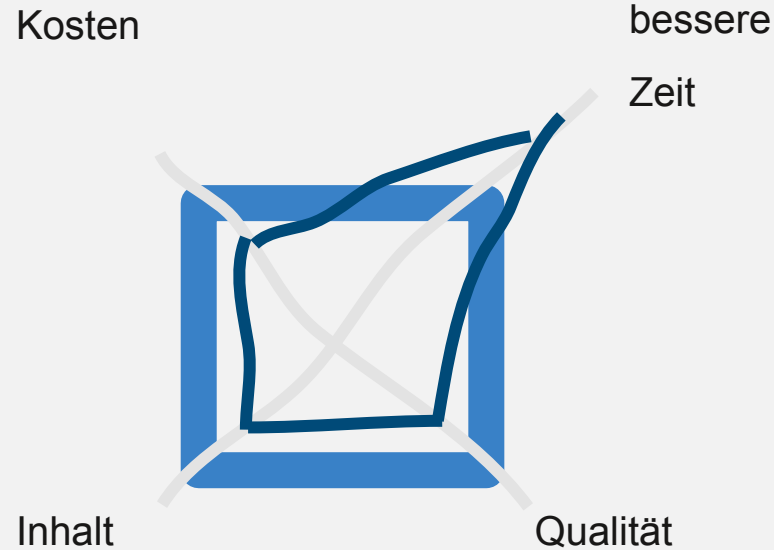


Das Spannungsfeld lässt sich auf unterschiedliche Weise beschreiben.

Das Teufelsquadrat nach Sneed



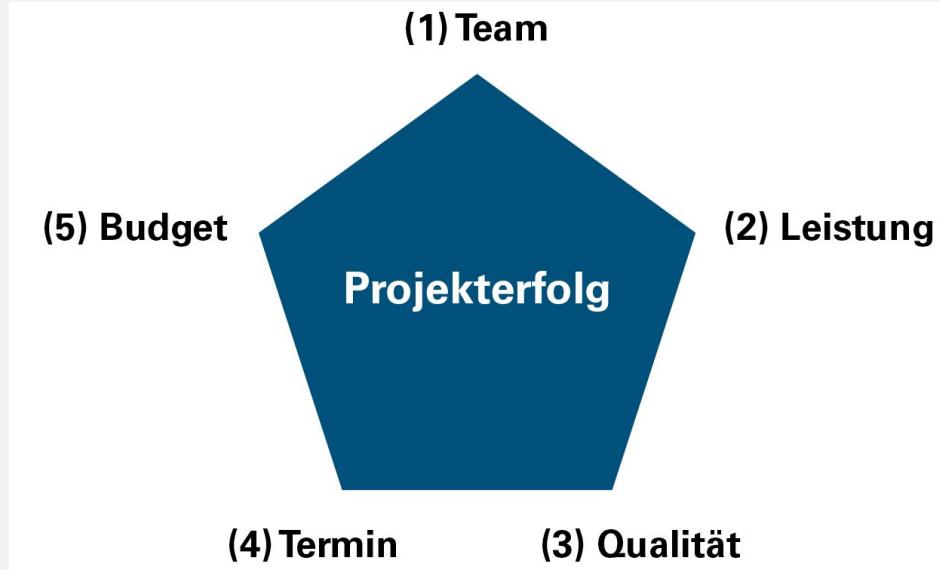
Das Spannungsfeld lässt sich auf unterschiedliche Weise beschreiben.



The QAware Way - das magische Fünfeck.



QA|WARE



Die Spannungsfelder helfen bei der Zieldefinition zu Projektstart.



QA|WARE

- Modelle unterstützen bei der groben Zieldefinition zum Projektstart:
 - Was wollen wir im Projekt erreichen?
 - Welche Qualitätsanforderungen stellen wir an unser Projekt?
- Darstellung der Schwachstellen:
 - Welche Kompromisse wollen wir im Projekt eingehen?
- Aufzeigen von Änderungen und Rahmenbedingungen:
 - Wie wirken sich Änderungen auf unsere Projektziele aus?
- Abfragen des aktuellen Projektzustands:
 - Wo stehen wir gerade in unserem Projekt?



QA|WARE

Was bedeutet “qualitativ hochwertig”?

Welches Fahrzeug ist qualitativ hochwertiger?



QA|WARE



Qualitativ hochwertig ist subjektiv.



QA|WARE

■ Schnelle Entwicklung

- Benutzer:innen brauchen schnell neue Features
- Schneller Live-Gang noch vor der Konkurrenz

■ Niedrige Entwicklungskosten

- Für Kund:innen mit wenig Budget
- Für mehr Gewinn der Softwarefirma

■ Schöner Sourcecode und saubere Architektur

- Für Entwickler:innen und Architekten
- Grundlage für alle anderen Qualitätsmerkmale!

■ performantes Programm

Qualitativ hochwertig ist objektiv.



■ Fehlerlosigkeit

- für Benutzer:innen, die durch Fehler gestört werden
- für Projektleiter:innen, die an der Zahl der Fehler gemessen werden
- für Entwickler:innen, die die Fehler beheben müssen

■ Viele Features

- Für Benutzer:innen
- Für den Vertrieb, der das Produkt so besser verkaufen kann

■ Benutzerfreundlichkeit

- Schnellere Aufgabenerfüllung

Qualitativ hochwertig bedeutet nicht: fehlerfrei.



QA|WARE

- Software ohne Fehler ist unrealistisch aufgrund:
 - Kosten/Nutzenverhältnis bei der Erstellung des Systems
 - Fehlerfreiheit setzt vollständig spezifiziertes Verhalten voraus
 - Vollständiger Test aller Programmpfade unmöglich
 - Korrektheitsbeweise sind nur für einfache System möglich

Ziel muss sein:

Kritische Ausfälle vermeiden. Die Kritikalität hängt dabei vom jeweiligen System ab.



QA|WARE

**Qualitativ hochwertig bedeutet:
“it depends!”**

Qualitativ hochwertig: it depends.



QA|WARE

Wie können wir entscheiden, welche Features und
Stellschrauben wir priorisieren?

Das Kano Modell hilft uns bei der Priorisierung der Funktionen unserer Software.



■ Im Kano Modell gibt es 5 Merkmale von Kundenanforderungen:

- Basismerkmale
- Leistungsmerkmale
- Begeisterungsmerkmale
- unerhebliche Merkmale
- Rückweisungsmerkmale

Basismerkmale sind Produkteigenschaften, die vorhanden sein müssen.



QA|WARE

- Müssen vorhanden sein, um die Grundfunktion des Produkts zu erfüllen
- Stellen aus Kundensicht selbstverständliche Eigenschaften dar
- Steigert nicht die Kundenzufriedenheit
- ABER: bei Nichterfüllung entsteht schnell Unzufriedenheit

Leistungsmerkmale werden als Qualitätsmerkmale bezeichnet.



QA|WARE

- Bieten die Chance, Kundenanforderungen besser als der Wettbewerb zu erfüllen
- Leistungsmerkmale werden häufig zum Vergleich bei Kaufentscheidungen herangezogen
- Leistungsmerkmale haben die Möglichkeit Kundenzufriedenheit zu steigern
- Kunden:innen haben in Bezug auf Leistungsmerkmale bereits Erwartungen, die bei Nichterfüllung zu Unzufriedenheit führen können

Begeisterungsmerkmale sind WOW-Faktoren.



QA|WARE

- Haben das größte Potential die Kunden:innen positiv zu beeinflussen
- Angebot überrascht, da vor dem Kauf keine spezifischen Erwartungen an diese Eigenschaft vorhanden waren
- Im klassischen Sinne mit dem Alleinstellungsmerkmal (USP) zu vergleichen
- Erzeugen großen Wettbewerbsvorteil
- Sind meist nur kleine Änderungen im Zusammenhang mit dem Gesamtprodukt
- Spielen die größte Rolle bei der Steigerung der Kunden:innenzufriedenheit und beim Aufbau des Wettbewerbsvorteils

Unerhebliche Merkmale sind meist nicht klar definiert.



QA|WARE

- Oft ist nicht klar, warum diese Merkmale vorhanden sind
- Spielen für die Zufriedenheit keine Rolle
- Spielen aber auch für die Unzufriedenheit keine Rolle
- Wenn sie nicht vorhanden sind, wird ihr Fehlen nicht bemerkt

Rückweisungsmerkmale führen direkt zu Unzufriedenheit.



QA|WARE

- Merkmale, die direkt zu Unzufriedenheit der Kunden:innen führen
- Schädigt oft das Markenimage der herstellenden Firma
- Sind Rückweisungsmerkmale nicht vorhanden, wird allerdings auch nicht die Zufriedenheit gesteigert

Produktmerkmale können nicht statisch sein.



QA|WARE

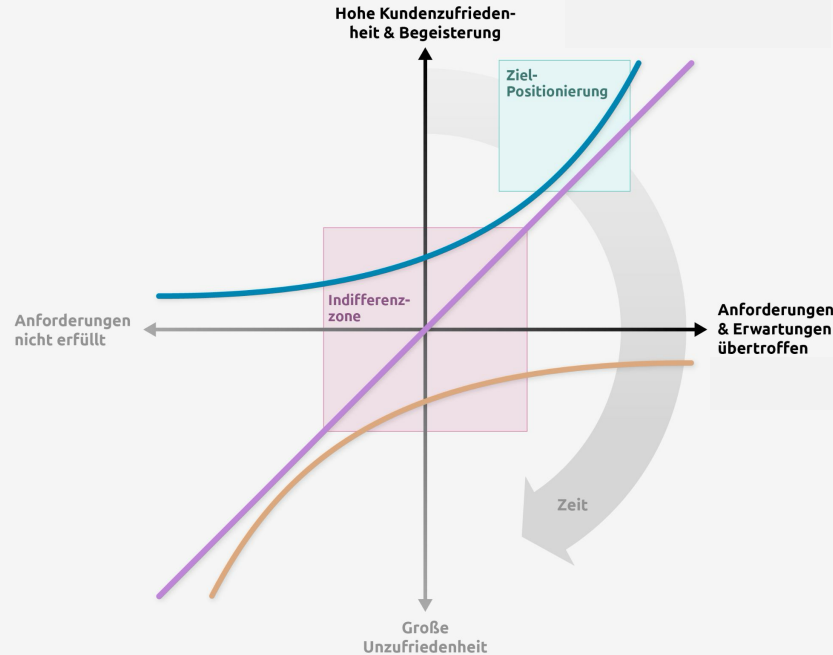
- Mit der Zeit entsteht bei den Kunden:innen ein Gewöhnungseffekt, so dass Begeisterungsmerkmale langsam zu Leistungsmerkmalen werden
- Leistungsmerkmale werden durch Konkurrenz, die ähnliche Features implementieren mit der Zeit zu Basismerkmalen

Das Kano Modell hilft uns bei der Priorisierung der Funktionen unserer Software.



QA|WARE

Das Kano Modell der Kundenzufriedenheit



Der Weg zu hoher Kundenzufriedenheit führt über drei Etappen.



- Das Produkt erfüllt Basismerkmale
- Im Wettbewerbsvergleich schneiden die Leistungsmerkmale gut ab
- In einem Bereich übertrifft das Produkt die Kundenerwartungen und begeistert → Begeisterungsmerkmale

Ziel: Positionierung im oberen, rechten Quadranten

Befindet sich das Produkt im Indifferenzbereich, wird keine starke Kundenbindung aufgebaut



QA|WARE

GQM als Tool zur Entwicklung eigener Qualitätsmodelle

Mit den Goal Question Metrics zu eigenen Qualitätsmodellen.



- Die Qualitätsmessung eines Projektes kann nicht für alle Projekttypen gleich sein
- Eine Banking-App hat beispielsweise andere Anforderungen an Qualität als ein Computerspiel
- Das Goal Question Metric Modell dient zur Entwicklung eines eigenen Qualitätsmodells, das genau auf die Anforderungen des Produktes zugeschnitten ist

GQM kann für jedes Projekt verwendet werden.



QA|WARE

- GQM ist nicht nur auf Softwareprodukte anwendbar
- Beschreibt die Verfahrensweise zur Erstellung eines Qualitätsmodells
 1. Charakterisierung des Projektumfeldes
 2. Formulierung von Fragen zur Zieldefinition
 3. Messziele identifizieren und Metriken ableiten
 4. Entwicklung von Mechanismen zur Datensammlung
 5. Daten sammeln, analysieren und interpretieren
 6. Erfahrungen zusammenfassen und anwenden

Zu Beginn definieren wir die Ziele des Projektes.



QA|WARE



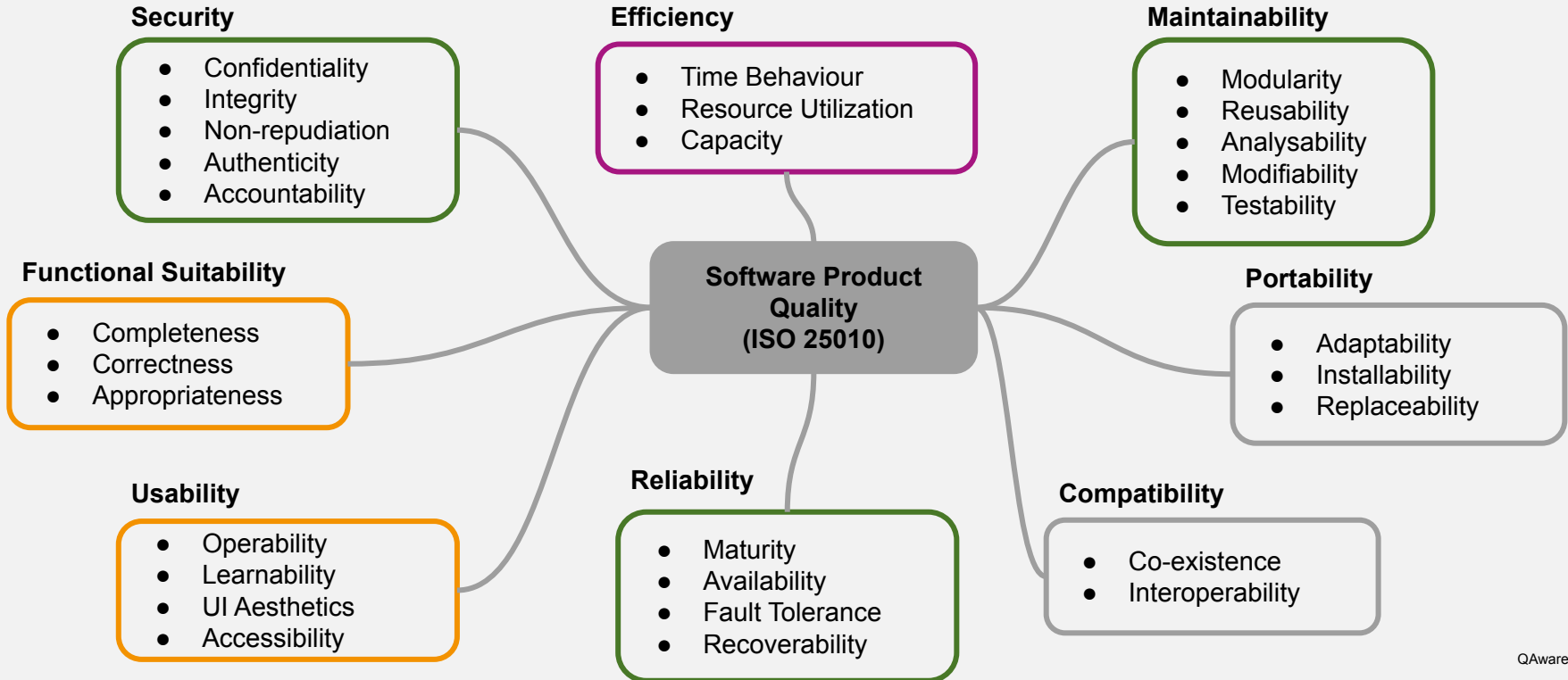
Goal

- Es gibt zwei Arten von Zielen: Geschäftsziele und Messziele
- Geschäftsziele sind langfristige Ziele der Unternehmung, sie treiben die Identifizierung von Messzielen an
- Jedes GQM Ziel sollte durch 4 Aspekte ausgedrückt werden:
 - Objekt der Messung
 - Zweck
 - Qualitätsfokus
 - Blickwinkel & Kontext

Der Qualitäts-Oktopus mit seinen “-ilities” hat Einfluss auf die 4 Sichten der Architektur eines Systems.



QA|WARE



Zu Beginn definieren wir die Ziele des Projektes.



QA|WARE



Goal

**Analysiere
zum Zwecke
in Bezug auf
vom Blickwinkel
im Kontext**

den Entwicklungsprozess
der Änderung
Korrektheit
des Entwicklers
des Projektes.

Zieldefinition für eine E-Learning Plattform.



QA|WARE



Goal

**Analysiere
zum Zwecke
in Bezug auf
vom Blickwinkel
im Kontext**

die Kursladezeiten
der Performancesteigerung
Geschwindigkeit
des Benutzers
der E-Learning-Plattform.

Dann definieren wir Fragen zur genaueren Definition von Zielen.



Question

- Messziele auf eine Menge an Fragen heruntergebrochen
- Was wollen wir über die Eigenschaften des Ziels lernen?
- Fragen müssen so gestellt werden, dass sie mit Metriken beantwortet werden können
- Kosten-/Nutzenverhältnis muss gegeben sein
- Nur vernünftige Anzahl an Fragen definieren

Abschließend identifizieren wir Messziele und leiten Metriken ab.



QA|WARE



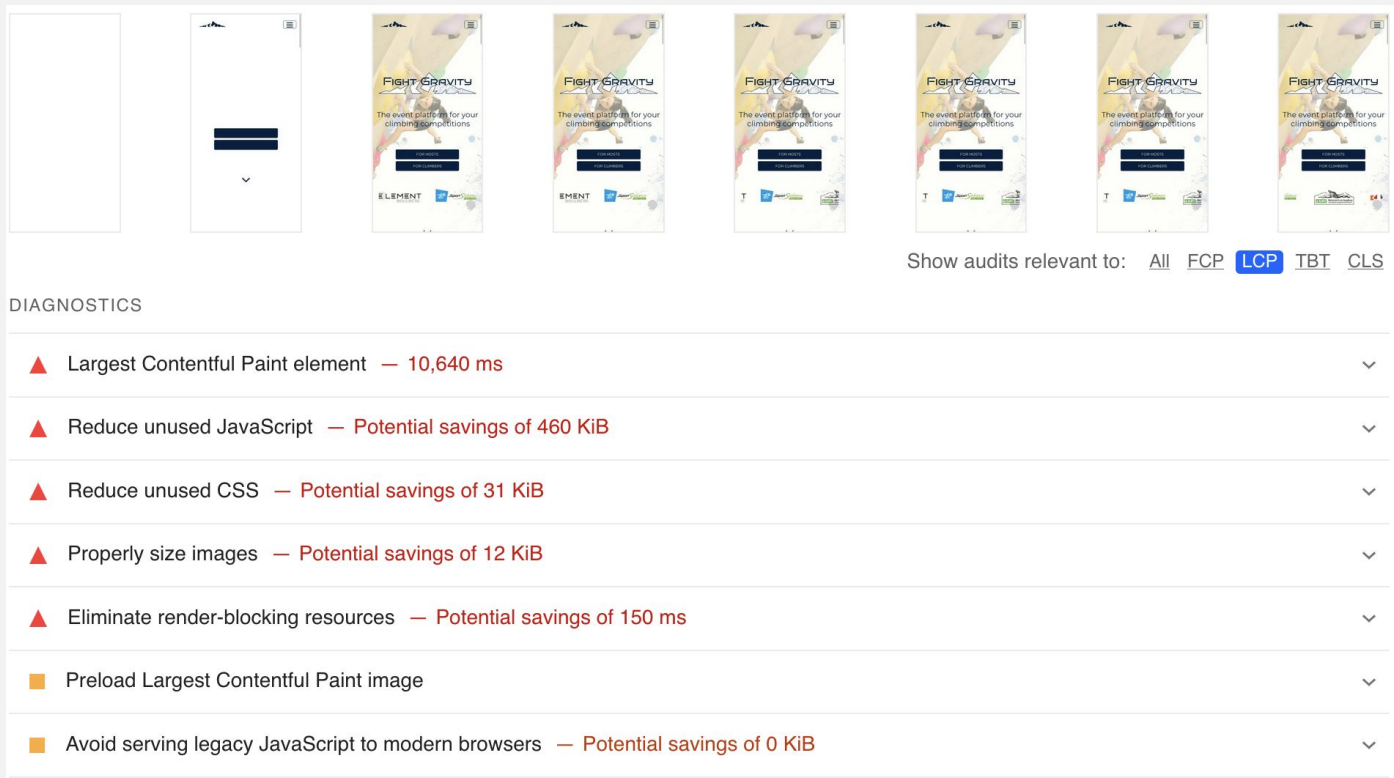
Metric

- Basierend auf den vorher definierten Zielen und Fragen werden Metriken abgeleitet
- Können sich direkt aus den Fragen ergeben, aber auch darüber hinausgehen
- Beispiele sind:
 - Vererbungstiefe
 - Methodenanzahl pro Klasse
 - Zyklomatische Komplexität

Zieldefinition für eine E-Learning Plattform.



QA|WARE



GQM ist ein dreistufiger Prozess.



QA|WARE



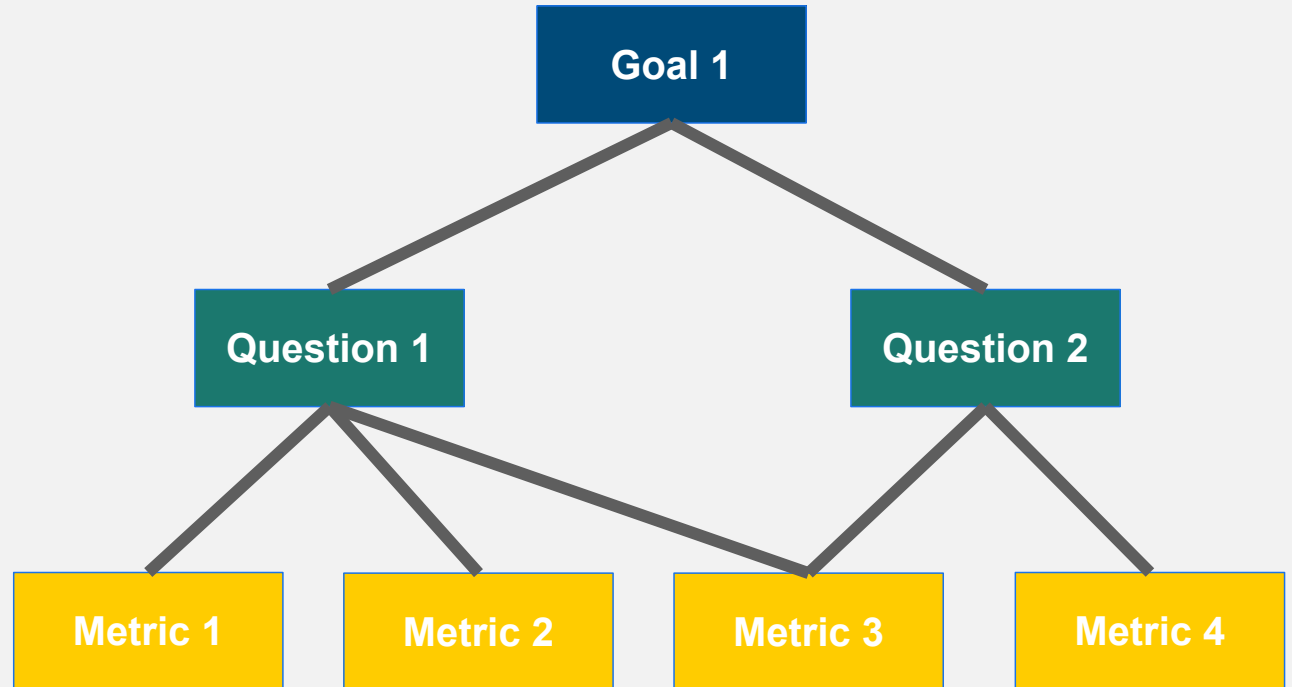
Goal



Question



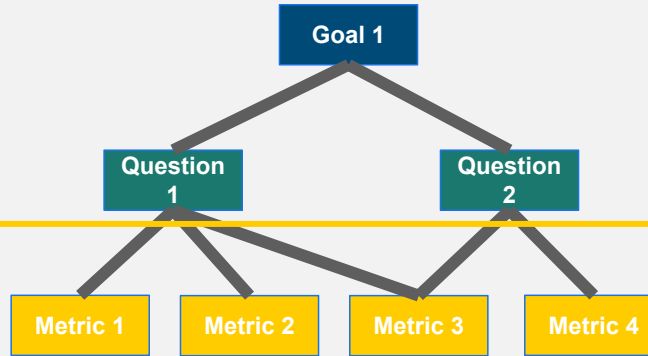
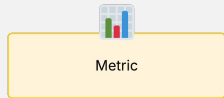
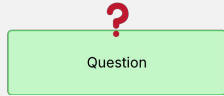
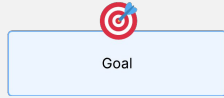
Metric



Auf Basis der Erkenntnisse können neue Ziele entstehen.



QA|WARE



Daten sammeln, analysieren und interpretieren



Erfahrungen zusammenfassen und anwenden

Quellenangaben zu Bildern



QA|WARE

- Folie 76: <https://www.muenchen.tv/passt-genau-massgeschneiderte-halle-fuer-den-airbus-a380-294117/>
- Folie 82: <https://www.functionize.com/blog/the-cost-of-finding-bugs-later-in-the-sdlc>
- Folie 102_1: <https://configurator.porsche.com/de-DE/model/992812>
- Folie 102_2: <https://www.mercedes-benz-trucks.com/de/de/trucks/unimog-offroad.html>
- Folie 115:
<https://www.fuer-gruender.de/wissen/existenzgruendung-planen/unternehmensstrategie/kano-modell/>